Sistemas Operacionais

História, Conceitos e Estruturas de Sistemas Operacionais

Rafael Descio, M.Sc.

Católica SC
Centro Universitário

Visão Geral

Dados da Disciplina

Carga Horária

60 Horas

45h Teórica / 15h Prática

Horários

4 aulas – Terças (2) e Quartas (1)

Interavalo: 20:20 (20 minutos)

Período

1º Período

Ementa

A disciplina aborda os fundamentos e a evolução dos Sistemas Operacionais, detalhando a estrutura básica de um sistema computacional e as funções essenciais de gestão de recursos. O conteúdo engloba o gerenciamento de programas e processos, noções de concorrência, escalonamento de CPU, administração de memória real e virtual, além de sistemas de arquivos e operações de entrada e saída. Também são introduzidos conceitos de segurança, proteção e uma visão técnica sobre sistemas operacionais modernos e virtualização.

Sobre o professor

Rafael Descio Trineto

Formação

Engenheiro da Computação

Pós-Graduação

Gerenciamento de Projetos de TI

PUC – PR (2014)

Mestrado Computação Aplicada

UDESC (2025)

Outras Atuações:

Gerente de Inovação Digital

Sobre

Sou Engenheiro de Computação, graduado pela Universidade Positivo (2008), com especialização Lato Sensu em Gerenciamento de Projetos de Tecnologia da Informação pela Pontifícia Universidade Católica do Paraná (2014). Sou Mestre em Computação Aplicada pela Universidade do Estado de Santa Catarina (2025), com pesquisa voltada às tecnologias de Blockchain, Tokens Não Fungíveis (NFTs) e Criptografia Homomórfica aplicada a sistemas distribuídos seguros e auditáveis. Possuo experiência acadêmica e profissional na área de Ciência da Computação, com ênfase em Engenharia de Software, atuando principalmente em desenvolvimento de software, sistemas de pagamentos digitais, metodologias ágeis e gerenciamento de projetos de tecnologia. Meus interesses de pesquisa incluem sistemas distribuídos, arquiteturas de blockchain, criptografia avançada, privacidade de dados e computação segura.

Objetivos da Aula de Hoje

- **Explicar** a necessidade histórica do surgimento dos Sistemas Operacionais.
- **Identificar** as principais fases evolutivas da computação.
- **Definir** o SO como uma máquina estendida e como um gerenciador de recursos.
- **Diferenciar** as diversas estruturas arquiteturais modernas.

ANOS 40

A Era Zero: O Programador como Sistema

Programação Física

O hardware era configurado manualmente via painéis de plugues e chaves. Não existia software de sistema.

Drivers Embutidos

Cada programa precisava conter todas as instruções para controlar o hardware. Não havia abstração.

Ineficiência Crítica

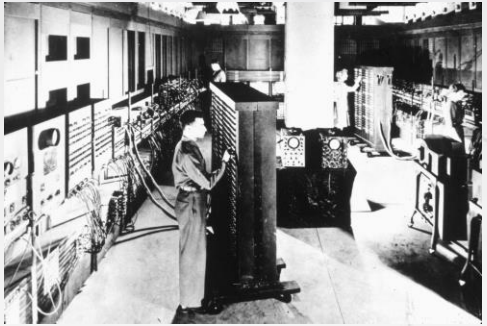
O tempo gasto preparando a máquina (setup) era muito maior do que o tempo de execução do cálculo.

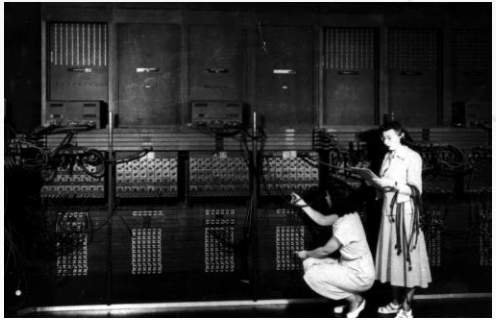
ENIAC (Electronic Numerical Integrator And Computer) (1945)

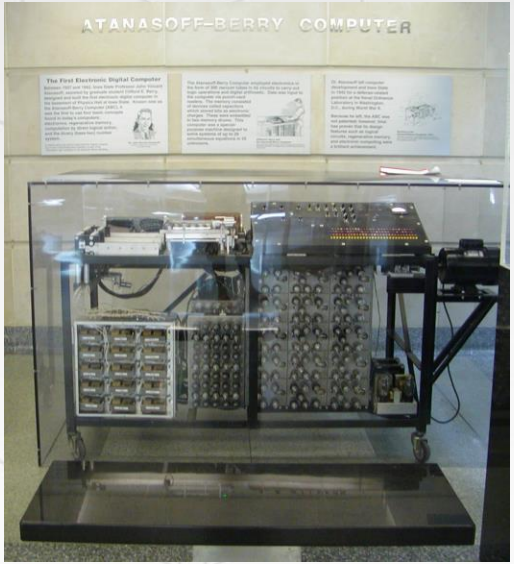
Era Pré-Sistema Operacional (1940–1955)

Contexto histórico:

- Programação manual
- Antes dos cartões perfurados
- Ausência de abstração
- Operadores humanos controlavam a máquina
- Uso de **válvulas eletrônicas**
- Alto consumo de energia
- Falha frequente de hardware







Era Pré-Sistema Operacional (1940–1955)

CONTEXTO

Programação **manual** direta no hardware via cartões perfurados e chaves físicas.

OPERAÇÃO

Um programa por vez. **Operadores humanos** controlavam o fluxo da máquina.

ABSTRAÇÃO

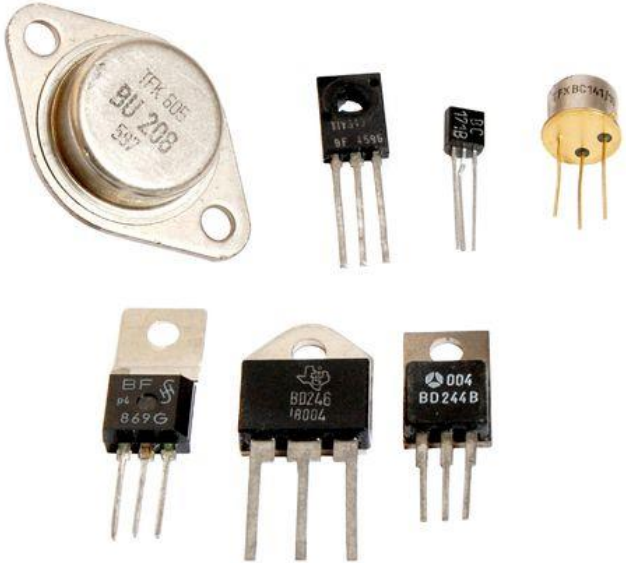
Inexistente. Nenhuma proteção de memória ou abstração de hardware.

EFICIÊNCIA

Baixa. A máquina ficava **ociosa** durante o setup manual entre tarefas.

GROOVE TUBES
PREMIUM
performance
matched
5 WHITE
GROOVE TUBES
EBJ

contra


BU 208
BD246
BD244B

2ª Geração: Sistemas em Lote

Contexto (Anos 50)

Cartões Perfurados

A introdução de mídias de armazenamento permitiu que os programas fossem preparados fora da máquina principal.

O Problema

Ociosidade da CPU

A CPU ficava parada enquanto o operador trocava manualmente as fitas e cartões entre um trabalho e outro.

A Inovação

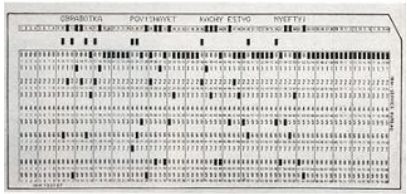
Monitor Residente

O primeiro "embrião" de um SO. Um software que ficava na memória e carregava o próximo trabalho automaticamente.

Objetivo

Processamento em Lote

Agrupar tarefas similares para execução sequencial, reduzindo drasticamente o tempo de setup.

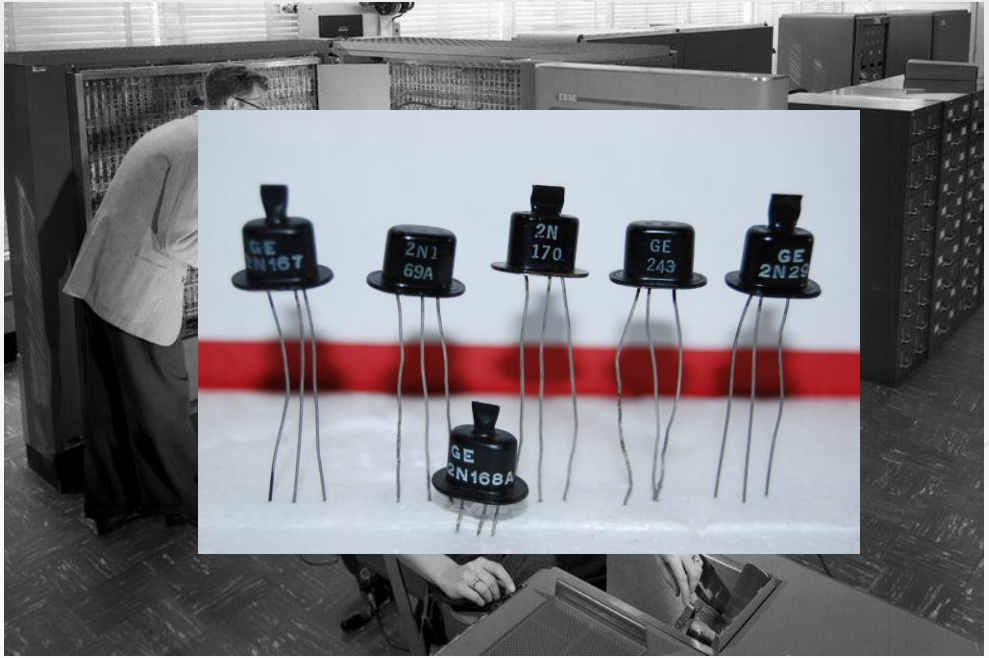


Um cartão perfurado de tradução russa com 80 colunas, datado de 1954.

AN/FSQ-7 (1955)

2ª Geração: Transistores

1. Uso de **transistores** → menor aquecimento e maior confiabilidade
2. Operados por especialistas
3. Entrada via **cartões perfurados**
4. Uso intensivo de **fitas magnéticas**

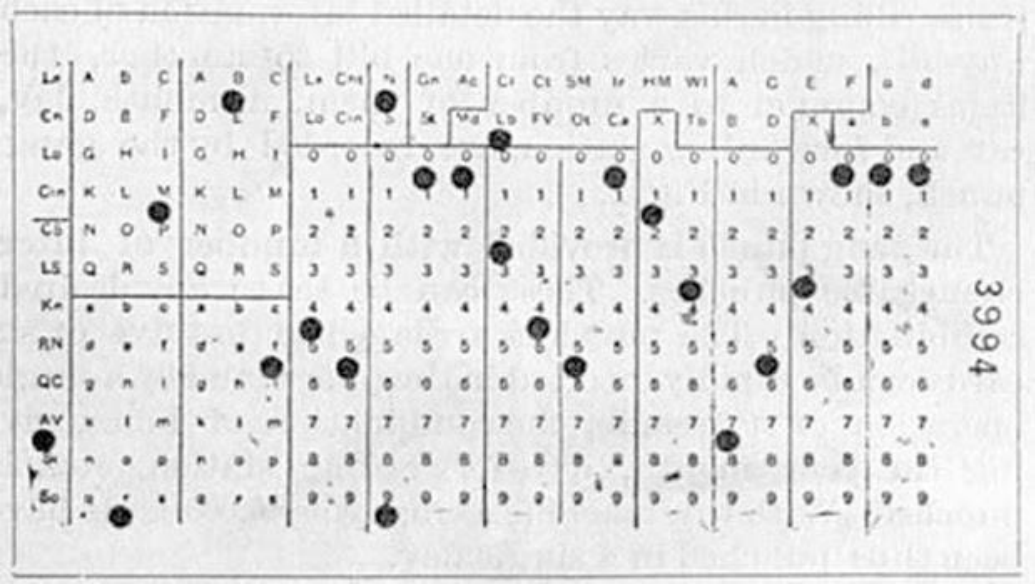


2ª Geração: 1955

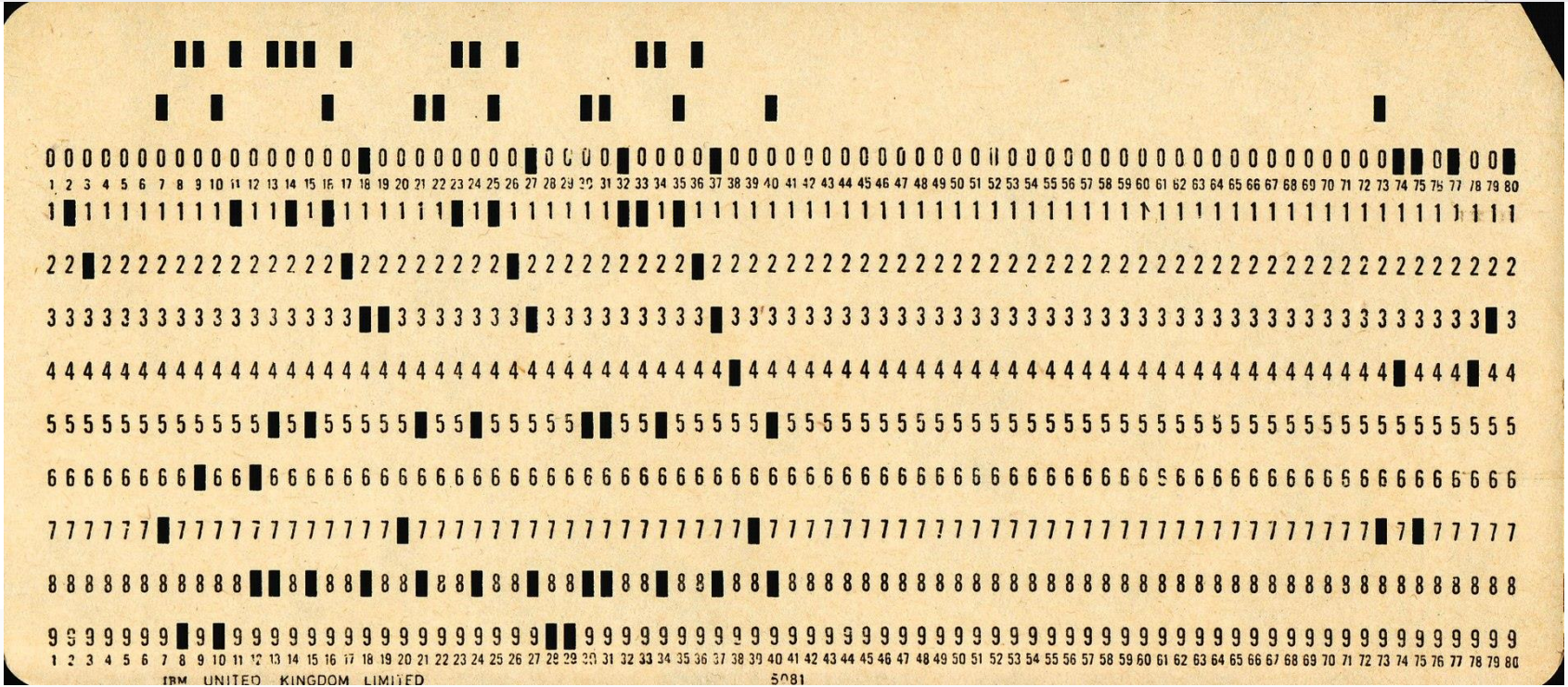
A programadora Madeleine Carey em 1955 com os 60.000 cartões que continham o programa de um sistema de defesa aérea corrido num computador IBM.

Era Pré-Sistema Operacional (1940–1955)

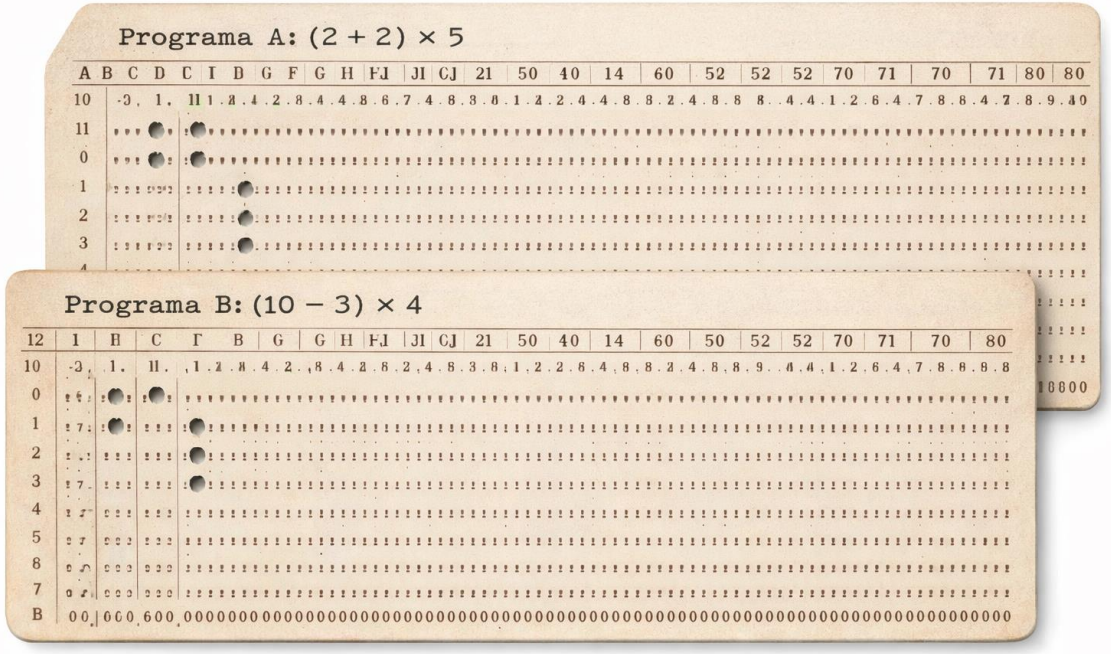
- 12 linhas por 24 colunas – Tamanho de uma nota de dólar.



Era Pré-Sistema Operacional (1940–1955)



Atividade prática



Dinâmica (15 minutos)

Dinâmica (15 minutos)

5 Alunos Candidatos

- 🧠 1 Programador
- 🎛️ 1 Operador da Máquina (Executa)
- 📦 1 Memória RAM (sim, uma pessoa será a memória!)
- 🕶️ 1 Gestor
- ⏱️ 1 Fiscal do Tempo – Gerencia o tempo e interrompe-o quando expira.

Dinâmica - Importante

- Operador só executa uma instrução por vez.
- Memória só guarda 3 instruções por vez.
- O gestor diz qual programa é mais importante
- Cada instrução não deve demorar mais de 30 segundos.
- Ao final de cada cartão, o valor vai pra memória.
- Toda vez que a Instrução “PRINT” é chamada, a memória é limpa.

Exemplo:

Gestor → Program A = (2 + 2 + 13) x 5

Programador → "Inicie o programa A"

Operador → "2 + 2 = 4"

Memória → "Guarda 4"

Operador → "4 (último valor) +13 = 17"

Memória → "Guarda 7"

Operador → "17 (Último valor) X 5 = 85"

Memória → "Guarda 85"

Operador → "Imprime 85"

Memória → "Limpa memória"

Coluna	Significado
10	LOAD
11	ADD
12	SUB
13	MUL
14	PRINT
0-9	Valor numérico
M	Memória

RESULTADO: 2+2 = 4

M	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
			O								O					
			O									O				

Coluna	Significado
10	LOAD
11	ADD
12	SUB
13	MUL
14	PRINT
0-9	Valor numérico
M	Último valor da Memória

RESULTADO: 13+4 = 17

M	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
		O									O					
				O							O					
O												O				

Coluna	Significado
10	LOAD
11	ADD
12	SUB
13	MUL
14	PRINT
0-9	Valor numérico
M	Último valor da Memória

RESULTADO: 5 x 13 = 85

M	**0**	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**11**	**12**	**13**	**14**	**15**
						O					O					
O														O		
															O	

Exemplo:

Gestor → Program A = (2 + 2 + 13) x 5

Programador → "Inicie o programa A"

Operador → "2 + 2 = 4"

Memória → "Guarda 4"

Operador → "4 (último valor) +13 = 17"

Memória → "Guarda 7"

Operador → "17 (Último valor) X 5 = 85"

Memória → "Guarda 85"

Operador → "Imprime 85"

Memória → "Limpa memória"

Simulando a realidade

Programa A – Folha de Pagamento

Calcular salário final:

- Salário base = 2.350
- Adicional = 450
- Imposto = 12%
- Resultado final?

Programa B – Estoque

Calcular valor:

- Quantidade = 180
- Valor Unitário = 35
- Desconto = 8%
- Resultado final?

Coluna	Significado
10	LOAD
11	ADD
12	SUB
13	MUL
14	PRINT
0-9	Valor numérico
M	Último valor da Memória

SIMULANDO 2.350 + 450

M	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15

Dinâmica (15 minutos)

5 Alunos Candidatos

- 🎥 1 Gestor – “Demanda externa” – Cria as chamadas
- 🧠 1 Programador (Lógica) – Cria as instruções.
- 🎛️ 1 Operador da Máquina (CPU) – Realiza os cálculos.
- 📦 2 Memória RAM (Guarda 3 instruções no máximo cada memória)
- ⏱️ 1 Fiscal do Tempo - “Kernel.”- Organiza os processos, gerencia o tempo e interrompe-o quando expira.

DINÂMICA COMPUTACIONAL SEM SISTEMA OPERACIONAL

COMPUTADOR ANTIGO (ERA PRÉ-SO)

= DEMANDA EXTERNA	= CÓDIGO / FORMA LÓGICA	= OPERADORA	= RAM	= RELÓGIO / INTERRUPÇÕES

Dinâmica - Importante

- Operador só executa uma instrução por vez.
- Memória só guarda 5 instruções por vez.
- O gestor diz qual programa é mais importante (no nosso caso, o estoque ou a folha de pagamento).
- Toda vez que a Instrução “PRINT” é chamada, a memória é limpa.
- Fiscal do tempo diz quem faz o quê. Quando o programador diz “Guarde o número 2 do programa B, o fiscal diz o recurso que vai ser usado”.

Simulando a realidade

Programa A – Folha de Pagamento

Calcular salário final:

- Salário base = 1.850
- Adicional = 250
- Imposto = 12%
- Resultado final?

Programa B – Estoque

Calcular valor:

- Quantidade = 210
- Valor Unitário = 20
- Desconto = 8%
- Resultado final?

A Crise da Eficiência (1955)

Problema Herdado da Era Pré-SO

- CPU extremamente cara e ociosa
- Tempo elevado entre execuções
- Troca manual de cartões e programas
- Dependência constante de operadores

Consequência

- **Baixo aproveitamento do hardware**
- **Custo por processamento muito alto**
- **Gargalo humano no ciclo de execução**

Surgimento do Processamento em Lote (Batch)

Monitor Residente

- Pequeno programa sempre carregado na memória
- Execução automática de uma sequência de jobs
- Introdução de fila de entrada (job queue)

Características Técnicas

- **Agrupamento de tarefas semelhantes**
- **Execução sequencial automática**
- **Redução da intervenção humana**
- **Primeiros conceitos de escalonamento**

2ª Geração: A Revolução da Multiprogramação

O Desafio (Anos 60)

O Descompasso Tecnológico

Com o surgimento dos transistores, a CPU tornou-se ordens de magnitude mais rápida que os dispositivos de entrada e saída (E/S). Em sistemas sequenciais, a CPU passava **90% do tempo ociosa** esperando o fim de uma leitura de disco ou fita.

UNIVAC 1108 (1964)

A Solução

Multiprogramação e Escalonamento

O Sistema Operacional passou a manter **vários programas na memória** simultaneamente. Quando um programa parava para esperar por E/S, o SO alternava a CPU para outro programa. Aqui nascem os conceitos fundamentais de **Gestão de Memória** e **Escalonamento de Processos**.

Sistemas Batch (1955–1965)

Surgimento do processamento em Lotes:

1. Fila de execução (job queue)
2. Operador separado do programador
3. Uso de fitas magnéticas
4. Primeiros mecanismos de escalonamento

Sistemas Batch & Multiprogramação

Sistemas Batch

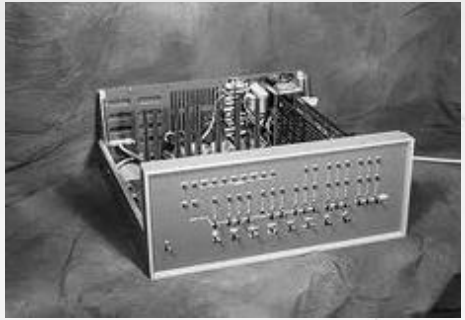
- **Processamento em Lote:** Agrupamento de tarefas similares.
- **Jobs Sequenciais:** Execução um após o outro sem intervenção.
- **Foco em Eficiência:** Redução do tempo de setup de CPU.

Multiprogramação

- **Memória Compartilhada:** Vários programas residentes ao mesmo tempo.
- **Alternância de I/O:** CPU troca de tarefa enquanto uma aguarda entrada/saída.
- **Escalonamento:** Introdução da gerência de prioridades e filas.

3ª Geração: ICs

1. Substituição de transistores discretos por ICs
2. Mainframes mais poderosos e confiáveis.
3. Terminais conectados a um único computador central
4. Surgimento do ***time-sharing***



Time-Sharing & Interatividade

A REVOLUÇÃO DO TERMINAL

Interatividade: Usuário interagem em tempo real.

Fatia o Tempo: Alternância rápida (Time Slice) entre processos.

SISTEMA & IMPACTO

Marcantes: CTSS, Mutics e Unix

Estrutura: Arquivos hierárquicos e gerenciamento de memória avançado.

Time-Sharing & Interatividade

1. Controle de processos
2. Gerenciamento de memória
3. Sistema de arquivos hierárquico

3ª Geração: Interatividade

Inovação Técnica

Tempo Compartilhado

O SO divide o tempo da CPU entre múltiplos usuários de forma tão rápida que cada um sente que possui a máquina inteira.

Fim da espera passiva por resultados.

Marcos Históricos

UNIX e Linguagem C

O nascimento de sistemas portáveis e modulares. O SO deixa de ser amarrado a um hardware específico.

Base para Linux e macOS modernos.

Revolução Social

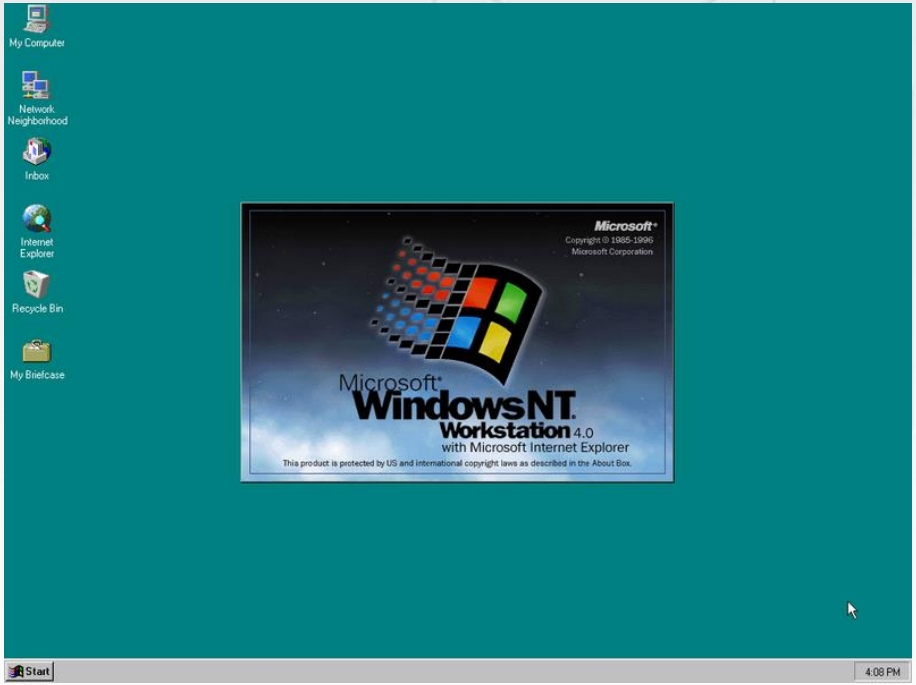
Computadores Pessoais

A computação sai das salas refrigeradas e entra nas casas. Foco total na interface e facilidade de uso.

O usuário torna-se o centro do sistema.

4ª Geração: PCs (1980 – até hoje)

1. Sistemas multiusuário e multitarefa
2. Interfaces gráficas (GUI)
3. Suporte a redes
4. Arquiteturas mais complexas (monolíticas, microkernel e híbridas)
5. Maior abstração de hardware
6. Proteção de memória e modos usuário/kernel
7. Sistemas portáveis e padronizados (POSIX)



5ª Geração: Dispositivos Móveis (2000 – até hoje)

1. Computação Distribuída e Móvel
2. O SO passa a ser plataforma de serviços
3. O conceito de máquina estendida evolui para infraestrutura estendida
4. Recursos são provisionados sob demanda
5. O kernel é apenas uma parte de um ecossistema maior (orquestração + virtualização + rede)

Hora de perguntar, dúvidas

O que é um Sistema Operacional?

1. MÁQUINA ESTENDIDA

Foco na Abstração. Transforma o hardware complexo em uma interface amigável.

- Arquivo em vez de setor de disco.
- Processos em vez de execução binária bruta.
- Esconde detalhes de baixo nível do programador.

2. GERENCIADOR DE RECURSOS

Foco na Eficiência. Atua como um árbitro para o uso do hardware.

- Gerencia CPU, Memória e Disco.
- Resolve conflitos entre programas.
- Garante proteção e compartilhamento justo dos recursos.

SO como Máquina Estendida

Dificuldade e Solução

O hardware “puro” é complexo, difícil de programar e pouco amigável.

O Sistema Operacional cria uma camada de abstração que:

- Esconde detalhes de baixo nível
- Simplifica o uso do hardware
- Oferece uma interface mais fácil para o programa

Objetivo

- Tornar o sistema utilizável
- Reduzir complexidade
- Aumentar produtividade do desenvolvedor
- Diminuir erros
- Padronizar acesso ao hardware

SO como Máquina Estendida

Analogia:

Imagine dirigir um carro.

Você:

- Pisa no acelerador
- Vira o volante

Mas não controla:

- Injeção eletrônica
- Combustão
- Sincronismo das válvulas

O **painel** e os **controles** são a **máquina estendida**.

O SO faz exatamente isso com o computador.

SO como Gerenciador de Recursos

Dificuldade

O computador possui recursos limitados:

- CPU
- Memória
- Disco
- Rede
- Dispositivos de E/S

Solução

O Sistema Operacional decide:

- Quem usa
- Quando usa
- Por quanto tempo usa
- Quanto pode usar

O SO é o árbitro do sistema.

SO como Gerenciador de Recursos

Analogia:

Pense em um aeroporto:

- Aviões querem pousar
- Pistas são limitadas
- Combustível é finito
- Portões são disputados

O controle de tráfego aéreo é o Sistema Operacional.

Chamadas de Sistema (System Calls)

A PONTE USUÁRIO-KERNEL

Programas não acessam o hardware diretamente por segurança.

- Solicitação formal de serviços ao SO.
- Exemplos: **read(), write(), fork()**.
- Padronização via APIs (ex: POSIX).

TRANSIÇÃO DE PRIVILÉGIO

O processador alterna entre estados de execução.

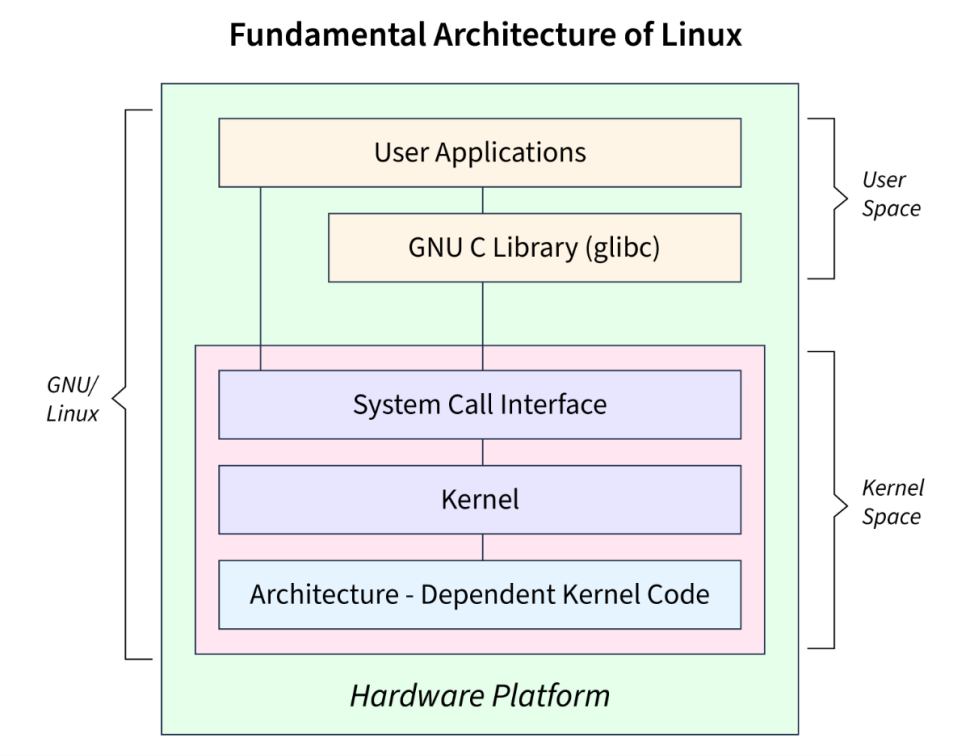
- **Modo Usuário:** Restrito, sem acesso direto ao hardware.
- **Modo Kernel:** Privilegiado, controle total da máquina.
- Garante a integridade do sistema.

Estruturas de Sistemas Operacionais

Sistema Monolítico (Monolithic)

- Tudo fica concentrado no Kernel
- Alto desempenho
- Baixo Isolamento

Exemplo: Linux, Android (Modular Monolithic)

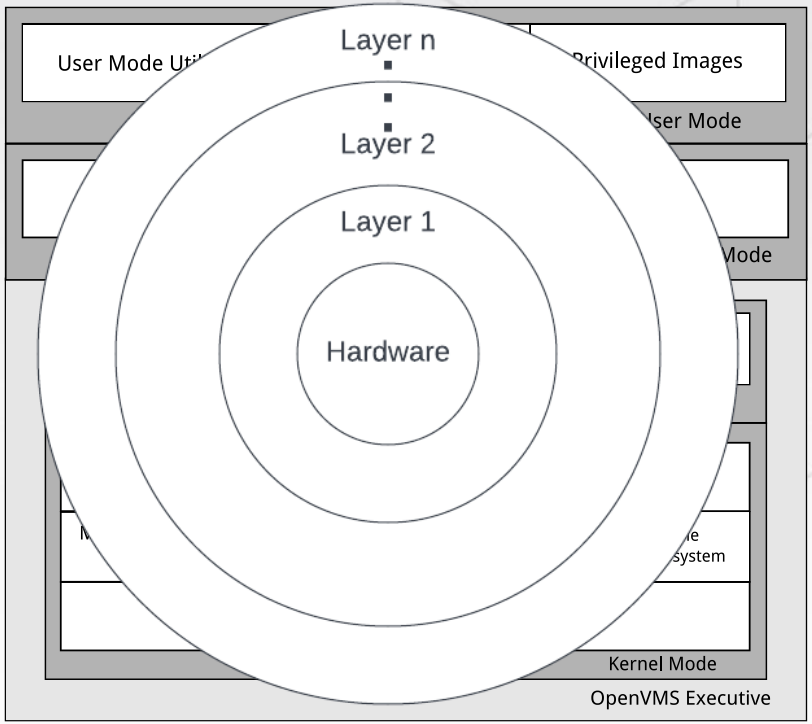


Estruturas de Sistemas Operacionais

Sistema em Camadas (Layered)

- Hierarquia de abstrações
- Organização modular
- Mais fácil de verificar formalmente

Exemplo: OpenVMS, MULTICS

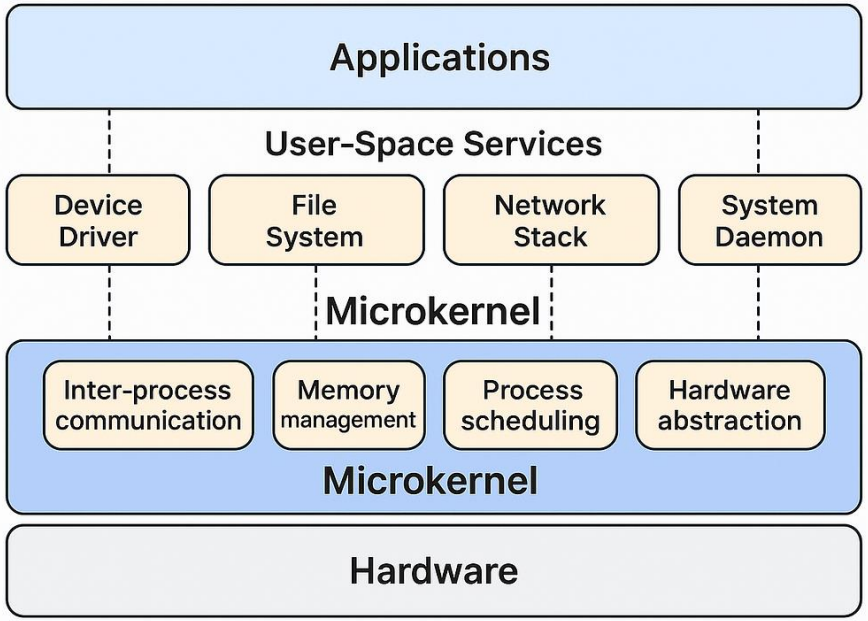


Estruturas de Sistemas Operacionais

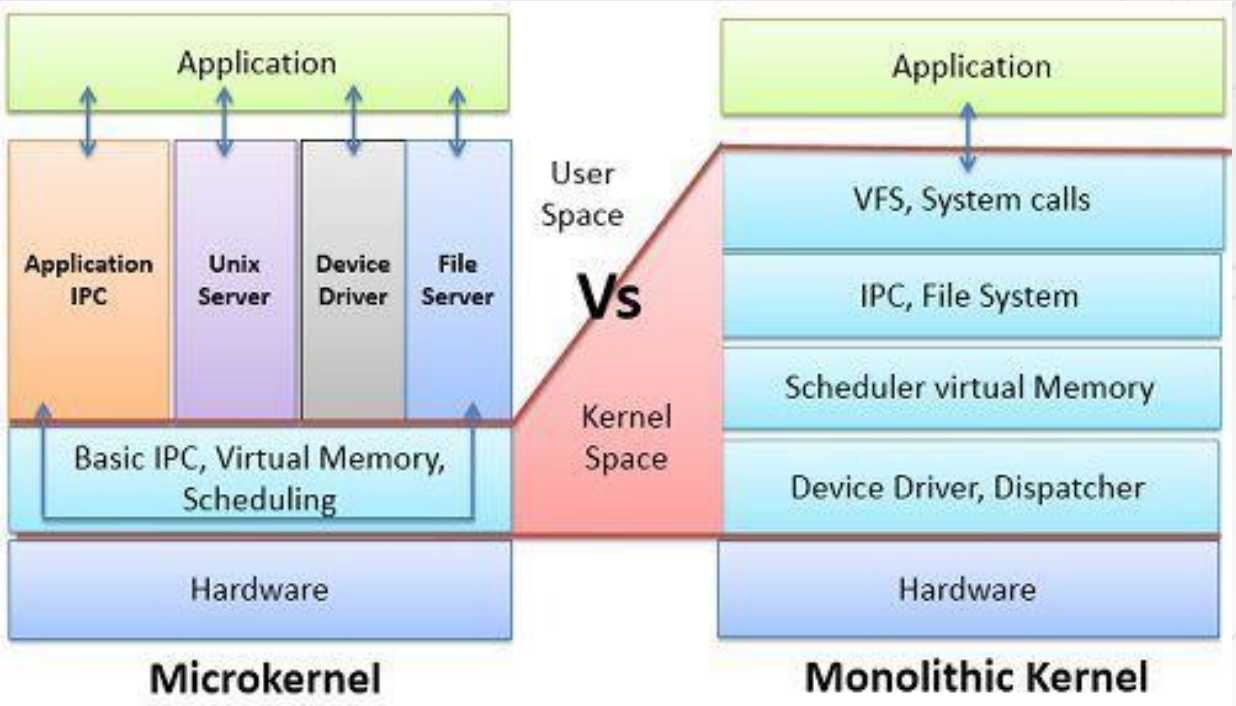
Sistema Microkernel

- Kernel mínimo
- Serviços em espaços de usuários
- Comunicação via IPC

Exemplo: MINIX, QNX, FreeRTOS (Arduino), dispositivos IoT e embarcados.



Comparativo Microkernel vs Monolitico



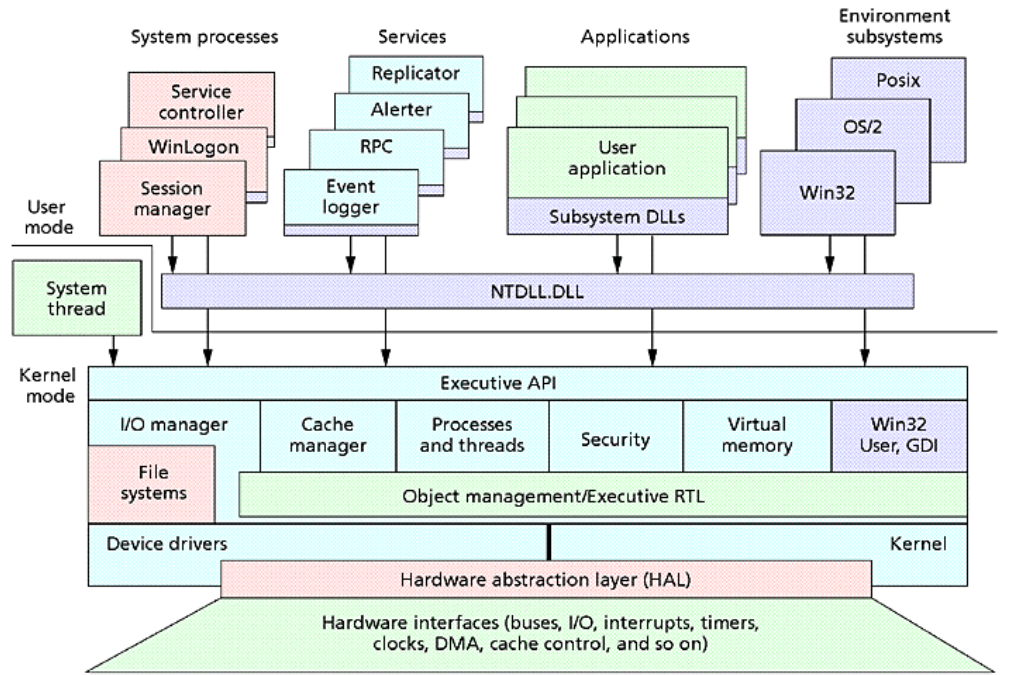
Estruturas de Sistemas Operacionais

Sistema Kernel Híbrido

Combinação de:

- Desempenho monolítico
- Modularidade do Microkernel

Exemplo: Windows, MacOS



Sistema Operacionais - Exercício 01

Primeiro exercício do semestre.

Distribuição dos **grupos** (5 alunos).

Avaliação: 0,5 ponto na Avaliação Teórica.

Lembrete, serão 4 pontos de Avaliação Teórica no Semestre.

Sistema Operacionais - Exercício 01

HISTÓRIA E ESTRUTURAS DE SISTEMAS OPERACIONAIS

Objetivo:
Analisar a evolução histórica dos Sistemas Operacionais e aplicar os conceitos de máquina estendida, gerenciador de recursos e estruturas arquiteturais, conforme abordagem apresentada.

Atividade:

Cada GRUPO ira simular a criação do seu próprio Sistema Operacional (imaginário, não precisa programar).

Referência:

Sistemas Operacionais Modernos, 5ª Edição, Tanenbaum e Bos (2022)

Sistema Operacionais - Exercício 01

Distribuição dos grupos (5 alunos):

( ) Grupo A – 1955 – Computador com válvulas e cartões perfurados

( ) Grupo B – 1968 – Mainframe com múltiplos terminais (time-sharing)

( ) Grupo C – 1985 – Computador pessoal com interface gráfica inicial

( ) Grupo D – 2026 – Dispositivo IoT agrícola conectado à nuvem

Sistema Operacionais - Exercício 01

Responder aos questionamentos:

- Necessidades do Sistema
- Estrutura Arquitetural
- Papel Principal do Sistema

Preencher:

- Identidade do Sistema
- Integrantes do Grupo

Hora de perguntar, dúvidas

Próxima aula

Conceitos Fundamentais de SO

- Aprofundar os conhecimentos de Máquina Estendida:
  - Arquivos em vez de blocos de disco.
  - Processos em vez de manipulação direta de CPU.
  - Memória virtual em vez de endereços físicos.
- Compreender o Gerenciador de Recursos:
  - Escalonamento
  - Alocação
  - Controle de concorrência

Aula 03

Próxima aula

Conceitos Fundamentais de SO

- Compreensão de Processos (Estado, registradores, Execução)
- Threads (Compartilhamento de memória, paralelismo)
- Concorrência (Múltiplos processos disputando recursos simultaneamente)
- Deadlock (Recursos bloqueados aguardando outros recursos)